

LICENSING OFFICER

 Review queue

LICENSING OFFICER

Review queue

3 need review · 0 waiting on operators · 4 decided

Refreshed just now · updates every 30 s



PermitFlow

Modernising regulatory licensing workflows

A working product for the Food Establishment Licence: submit, review, feedback, resubmit and decide, with an advisory document check.

Needs review 3

Waiting on operator 0

Decided 4

All 7

REFERENCE

BUSINESS · APPLICANT

STATUS · CHECKS

PF-2026-001009

Revision 1 · 2 open

Kopi & Kaya Toast House Pte. Ltd.

Tan Wei Ling · 10 Jalan Besar #01-12 · just now

● Under Review

just now

● 2 to check

Submitted 19 Sep 2026

Continue review →

PF-2026-001001

Revision 1

Scenario Kopi House Pte. Ltd.

Tan Wei Ling · 10 Jalan Besar #01-12 · 14 h ago

● Under Review

14 h ago

● 4 to check

Submitted 19 Sep 2026

Continue review →

PF-2026-001000

Revision 1

Scenario Kopi House Pte. Ltd.

Tan Wei Ling · 10 Jalan Besar #01-12 · 19 h ago

● Under Review

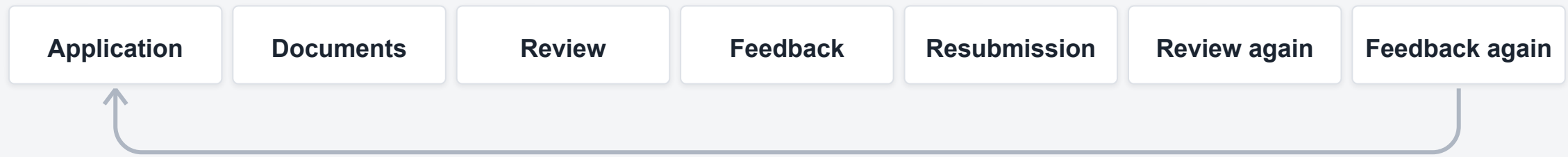
19 h ago

● 4 to check

Submitted 18 Sep 2026

Continue review →

Licence applications go round in a slow, lossy loop, and each turn costs both sides time and trust.



- 01 Incomplete or incorrect submissions cause repeated rejection cycles.
- 03 The whole application is re-entered on resubmission and data is lost between rounds.
- 05 The same comments are rewritten for the same common problems.

- 02 Feedback is not tied to a section or a document, so the business does not know what to fix.
- 04 The officer cannot see what changed, or whether an earlier issue was actually resolved.
- 06 Applications get lost between stages and the record of what happened is incomplete.

Before: a pile, a letter with no reason, the whole thing again, and an officer comparing by hand.



1

The pile

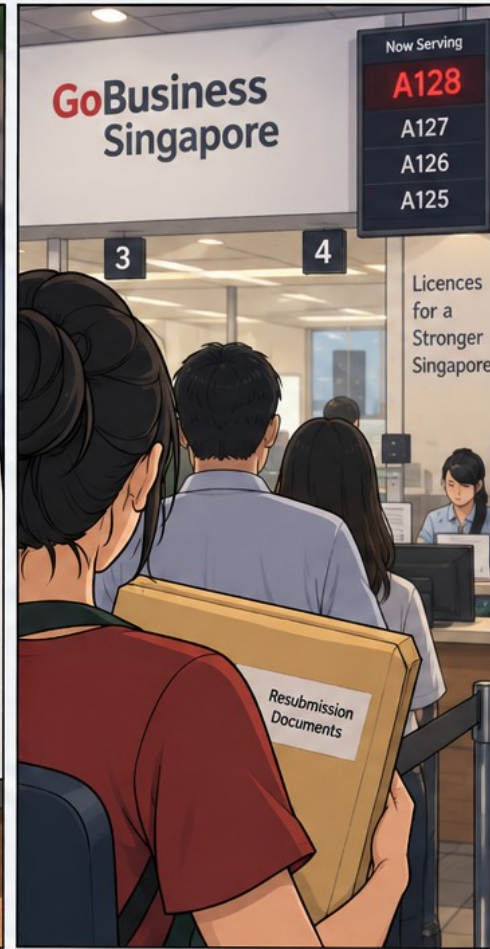
Incomplete documents and months of waiting.



2

The letter

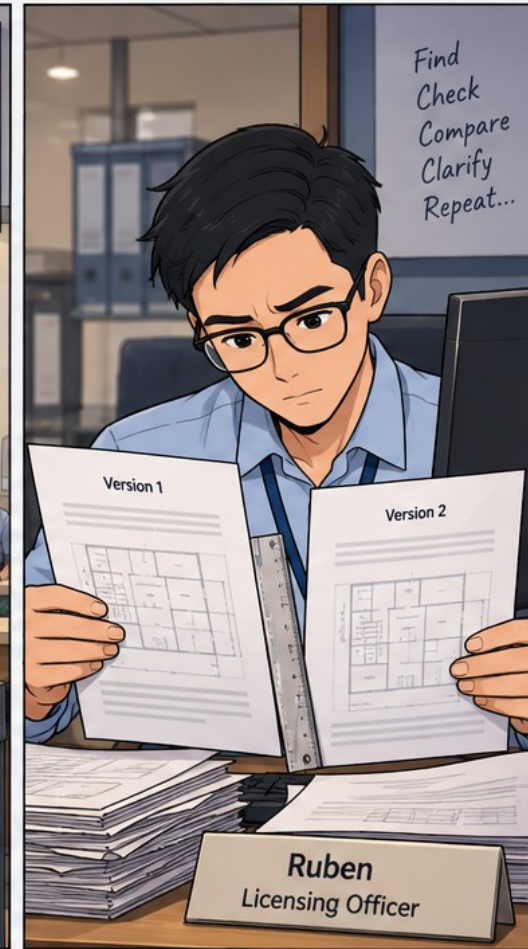
The application is returned as incomplete.



3

Everything again

Queue up and submit everything again.

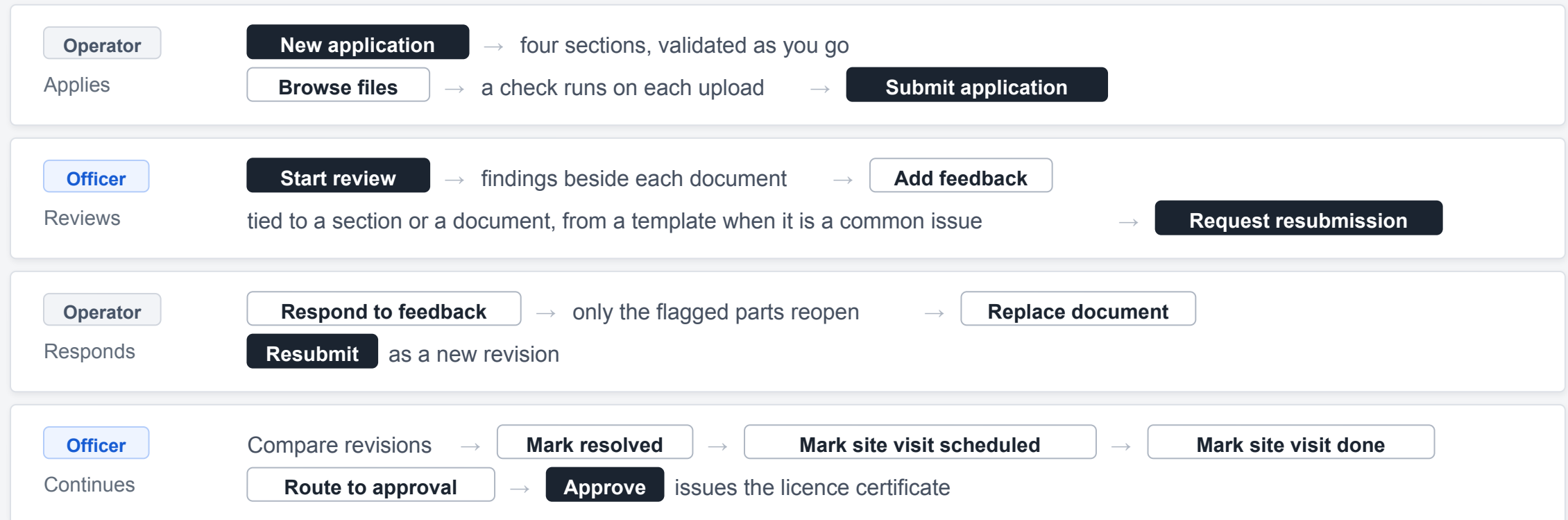


4

The officer's hunt

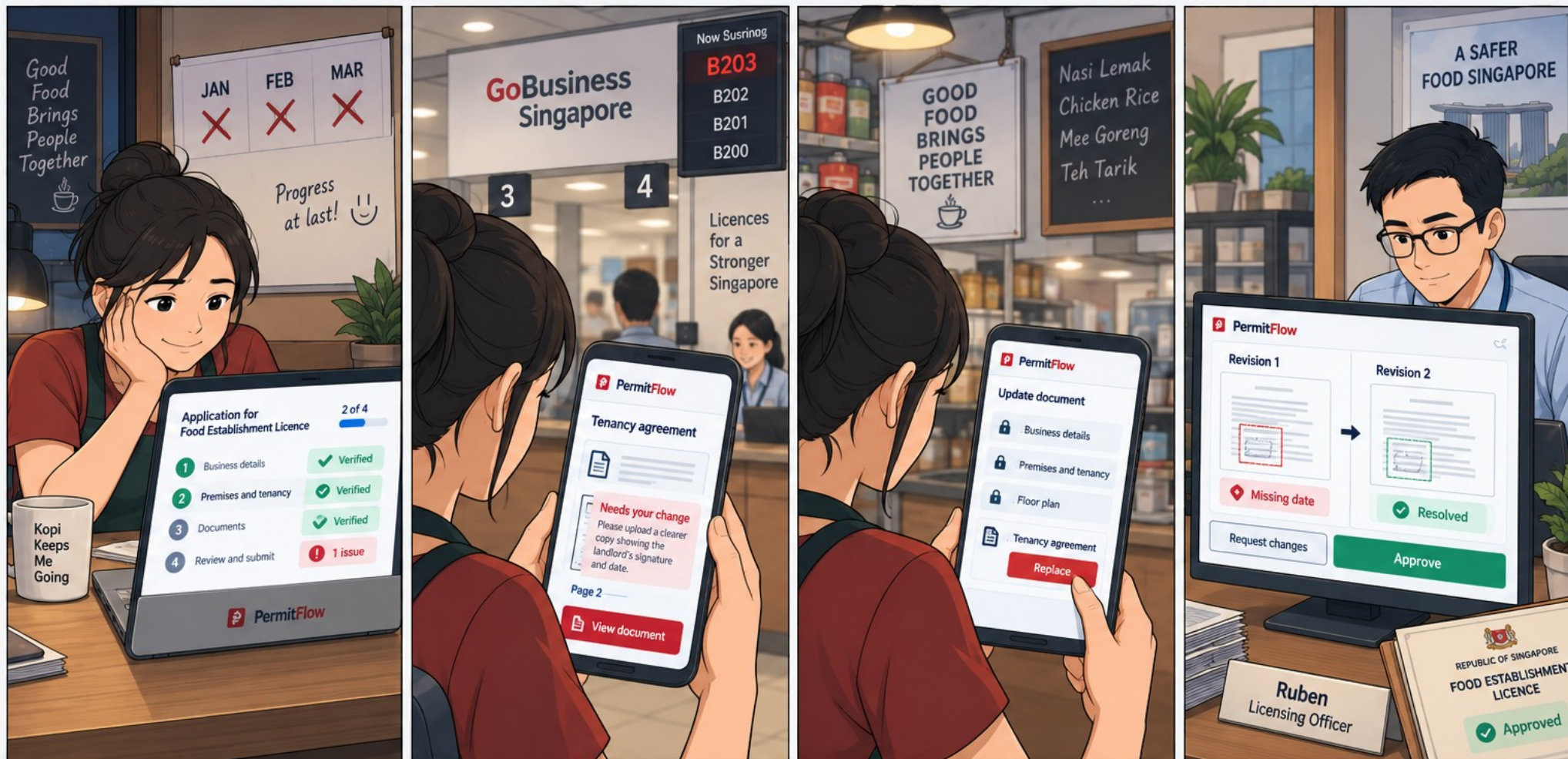
Comparing documents to find what's missing.

One continuous workflow: submit, review, feedback tied to a section or a document, then resubmit only what was flagged.



Every dark or outlined chip is a control label in the product, spelled as it appears.

After: the same four moments with PermitFlow, and each one is shorter.



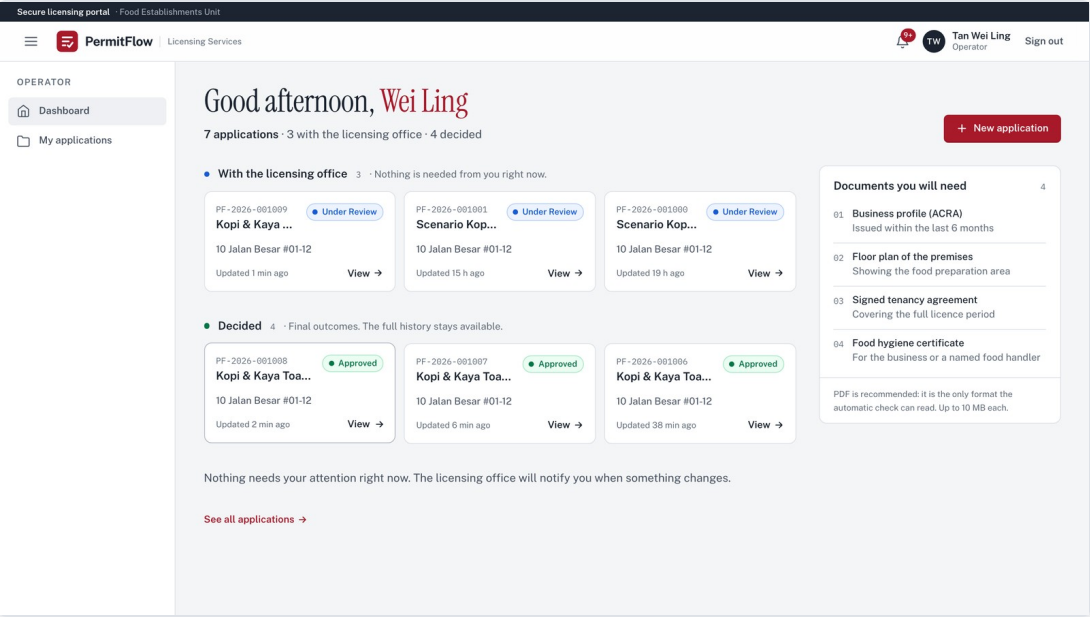
1 A clearer path
See your progress at a glance.

2 Clear, specific feedback
See exactly what to fix.

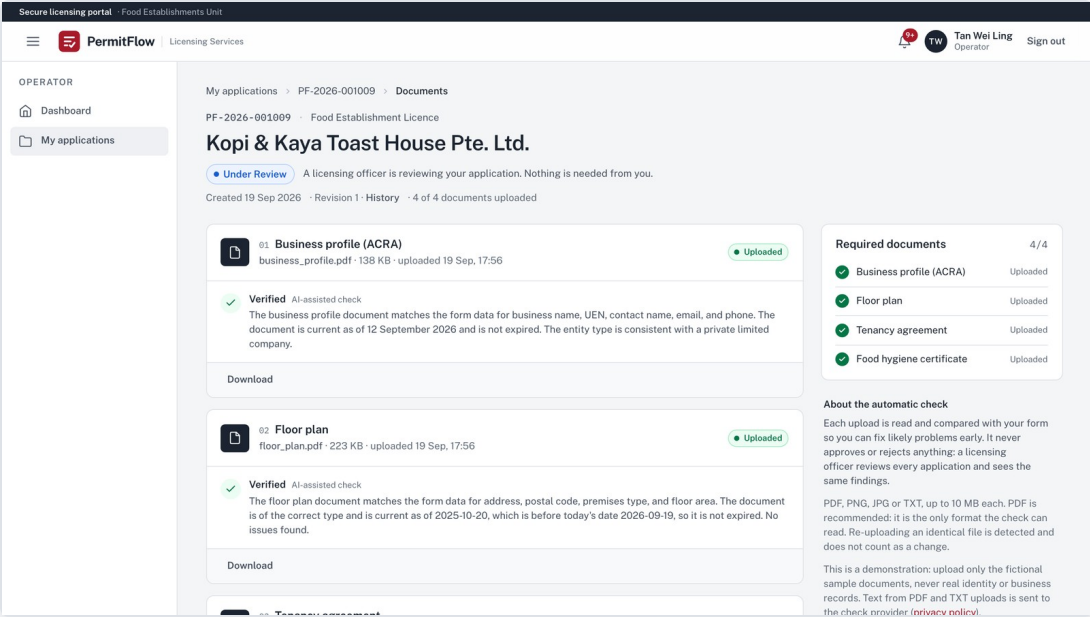
3 Only what's needed
Update just the flagged part.

4 Ready for approval
Issues resolved, licence within reach.

The operator's side: applications grouped by who needs to act, and a check result on every upload.

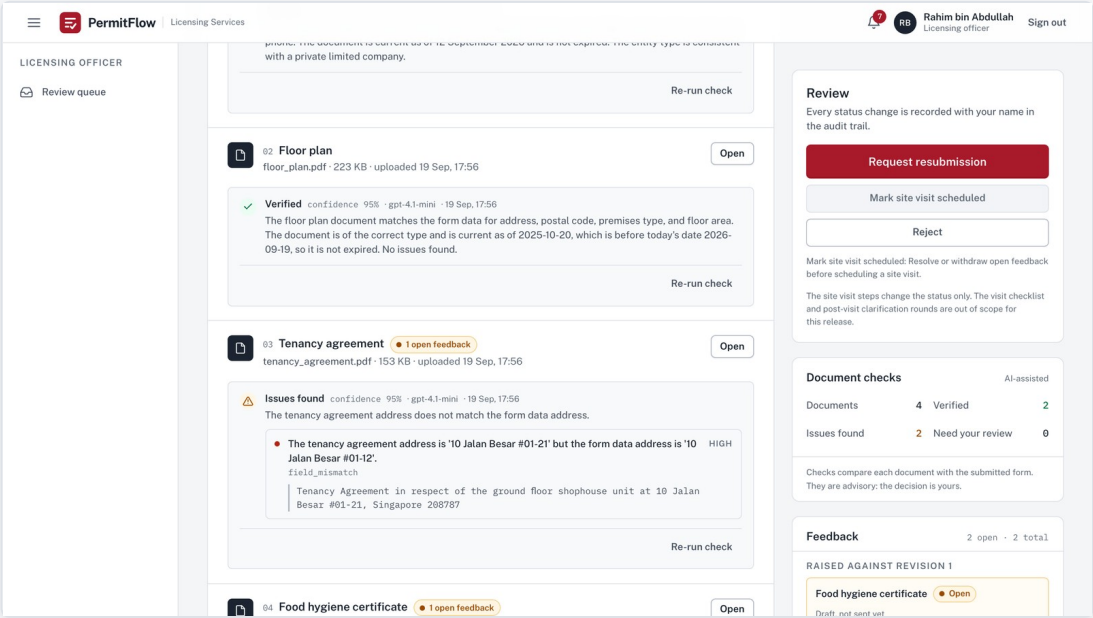


Operator Dashboard: applications grouped by who needs to act

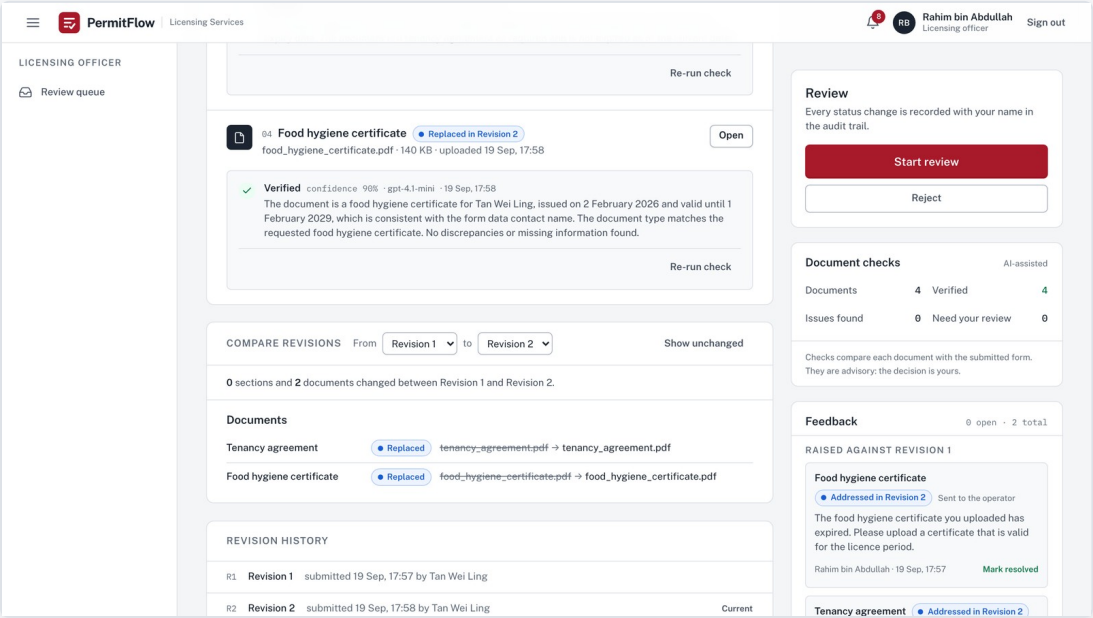


Operator Documents: a check result on each upload, in plain language

The officer's side: findings with evidence beside each document, and a compare view for every round.



Officer Case: findings with evidence beside each document, feedback rail on the right



Officer Compare revisions: what changed, and each item's resolution

The AI assists. Humans decide.

Per upload, before submission

Advisory check on every document

- Extract the text; unreadable files are reported without a model call
- Compare the document with the form: name, UEN, address, dates
- Flag issues with a closed issue code
- Quote the evidence from the document
- Self-reported confidence; below 0.6 becomes Needs your review

What the officer sees

Officer

- Outcome: Verified, Issues found, Needs your review
- Each issue with its code, severity and evidence quote
- Confidence and model on the card; prompt version in the audit event and the trace

Re-run check at any time

What the officer decides

Every decision is a person's, and audited

- The findings never change a status or write feedback
- The Approve dialog warns about unresolved findings but never blocks
- The state machine has no AI actor

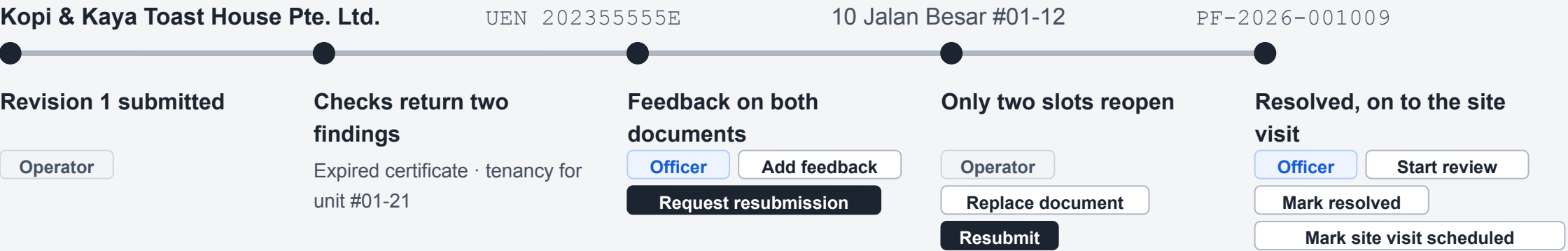
Deterministic by design, not AI

Never touched by the model

- Feedback templates
- Revision compare
- Status changes
- The prompt-injection rule, applied before the verdict is read

Operator sees the outcome, a plain explanation and each issue in plain words. **Never the evidence quote, never the confidence.**

Two planted problems, one feedback round, and the record shows exactly what changed.



DOCUMENTS AND CHECK RESULTS

Business profile (ACRA)

business_profile.pdf

● Verified

Floor plan

floor_plan.pdf

● Verified

Tenancy agreement

Replaced in Revision 2 · tenancy_agreement.pdf

● Verified

Food hygiene certificate

Replaced in Revision 2 · food_hygiene_certificate.pdf

● Verified

FEEDBACK RAISED AGAINST REVISION 1

Food hygiene certificate

Template: Food hygiene certificate has expired

● Resolved

Tenancy agreement

The tenancy agreement names unit #01-21, but the application and the business profile give #01-12

● Resolved

Compare revisions

From Revision 1 to Revision 2

0 sections and 2 documents changed

Status

● Site Visit Scheduled

Six principles, each enforced by code.

01 **Continuous workflow**

One application, any number of rounds. Fourteen states with actors, guards and labels as a table; every status change goes through one function.

02 **Immutable revisions**

Every submission is a snapshot. The compare view is a read-time diff, computed and never stored, so nothing is lost between rounds by construction.

03 **Contextual feedback**

Every item is tied to a section or a document. "Only the flagged parts" is an authorization rule: the server answers 403, the UI walks the operator through exactly those targets.

04 **AI-assisted, never AI-decided**

The verifier writes only to `verification_runs`. The state machine has no AI actor. Injection is handled by a deterministic rule before the verdict is read.

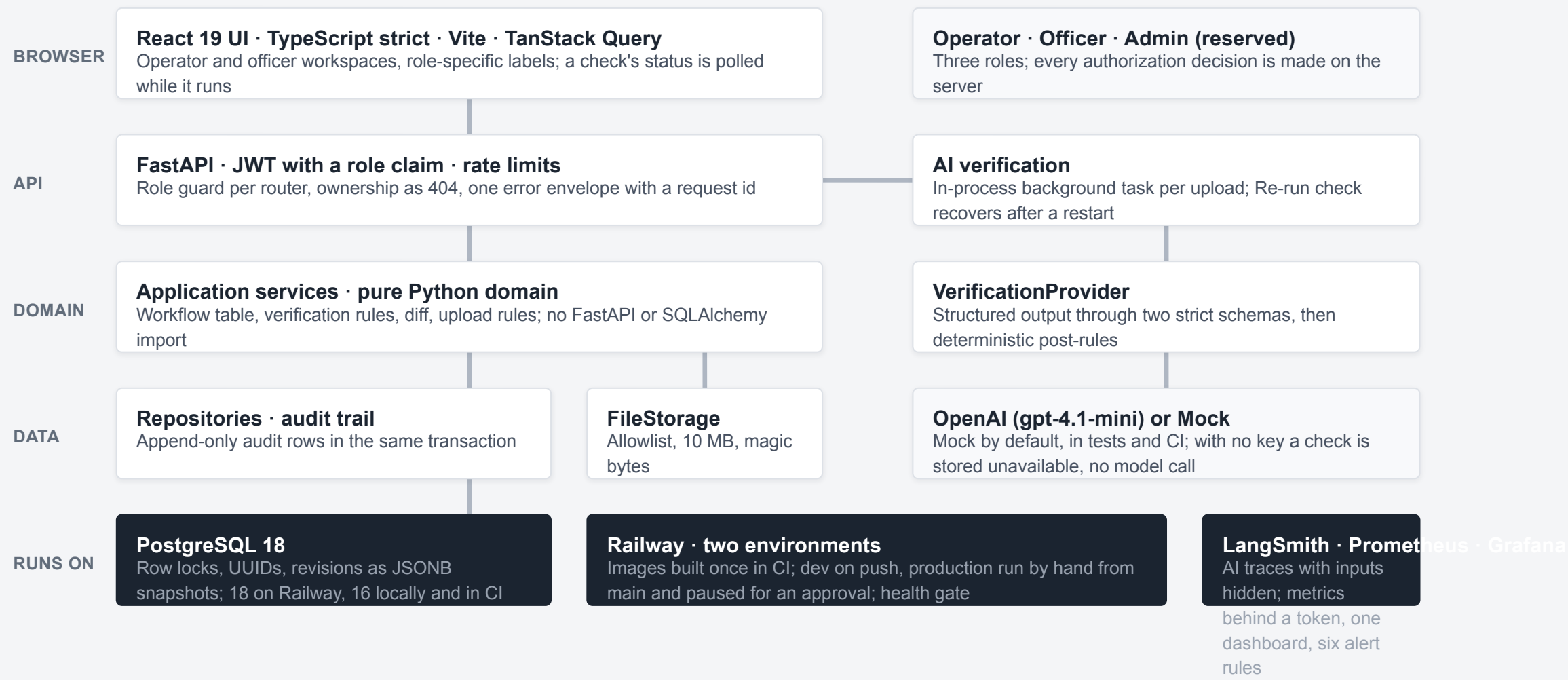
05 **Same-transaction audit**

A status change, its audit row, the operator's notification and, on approval, the licence certificate either all happen or none do.

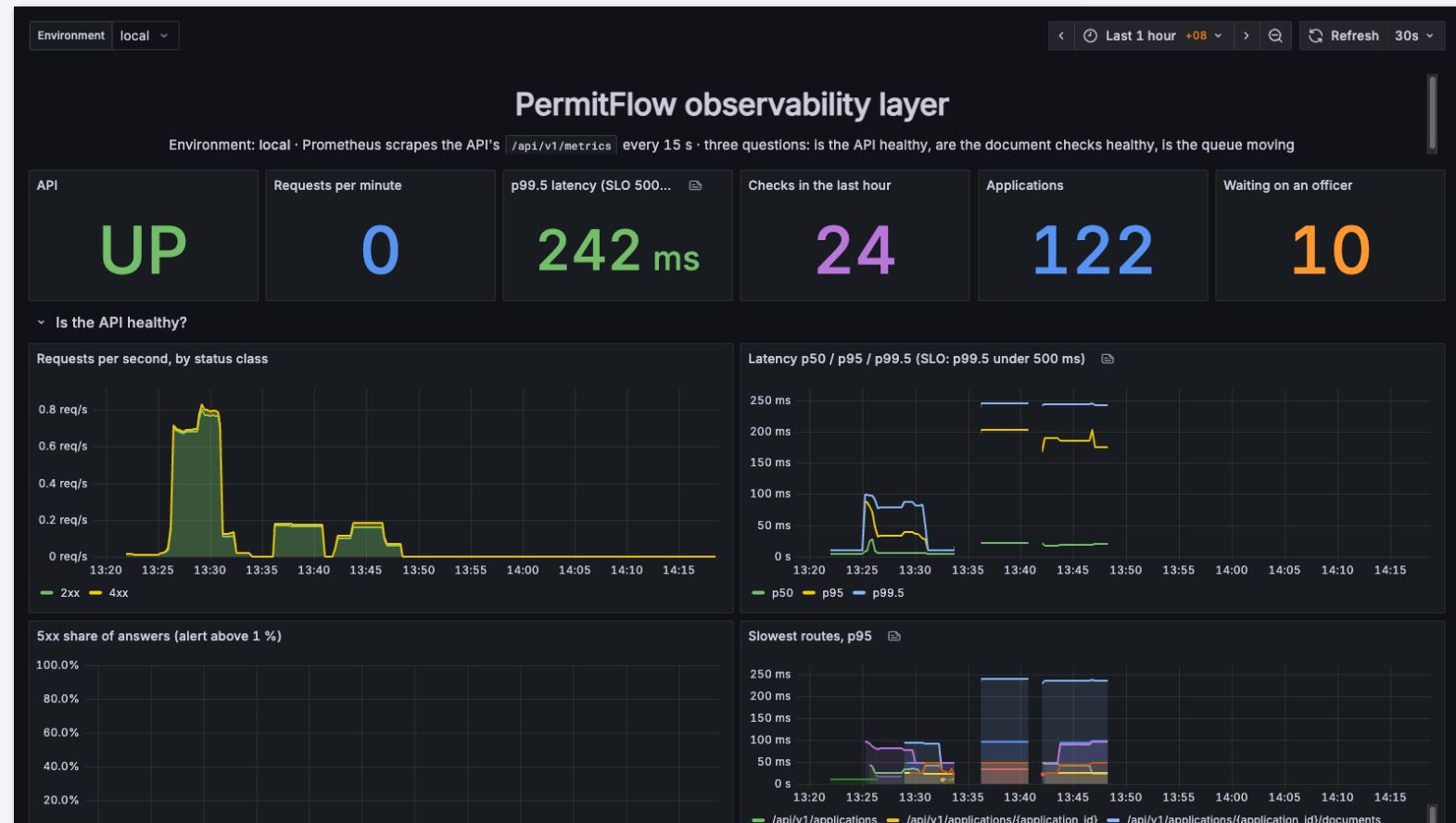
06 **Role-aware by construction**

Authorization is server-side on every route with an authorization test per endpoint. Operators receive their own labels and never an internal status code.

A modular monolith with an explicit state machine, an advisory AI module and an audit trail written in the same transaction.



One dashboard answers whether the API is healthy, whether the document checks are healthy and what they cost, and whether the queue is moving.



Grafana on grafana.dev.permitflow.space, fed by Prometheus scraping the API's token-protected metrics every 15 seconds; six alert rules on the service objective and an hourly digest reach Telegram, where a bot also answers `/status`; the dashboard is a generated file in the repository.

Checked by tests, gates and a written record.

763 backend tests **160** frontend tests **8** Playwright specs **95 % and 81 %** coverage **588** state-machine combinations **23** threats **74** stories, 11 deferred

- ✓ **Tests.** Backend on a real PostgreSQL; Playwright journey and scenarios in CI against the full stack.
- ✓ **State machine.** 588 transition combinations over 14 states and 3 actors; every invalid one rejected.
- ✓ **Authorization.** Server-side on every route; an authorization test for every application-scoped endpoint; ownership answered as 404.
- ✓ **Validation.** Strict schemas at the HTTP boundary and on the model's output; one error envelope with a request id.
- ✓ **Audit.** Append-only rows in the same transaction as the change; actor and time on every event.
- ✓ **CI gates.** Lint, types, an 80 % coverage floor that fails the build, the six-stage AI gate, accessibility, secret and dependency scans.
- ✓ **AI evaluation.** 14 golden cases and 21 fairness runs, 18 of them name-swapped, on the live model; nightly and on change.
- ✓ **Delivery.** Images built once; production behind an approval; health gates after every deploy; request ids everywhere; Prometheus metrics and a Grafana dashboard with six alert rules.

Known gaps first, then what production would need.

01 First: use case 3, the site-visit checklist

Statuses and transitions exist and are tested; the checklist, capture screen and clarification round are not built.

03 Object storage with a virus scan

S3-compatible storage with signed URLs behind the existing storage interface.

05 Identity

Registration, password reset, MFA and SSO; seeded accounts today.

07 AI evaluation and assurance

A labelled set from officer overrides, broader red-teaming, calibration, multilingual injection defence.

09 Delivery, backups and alert routing

Scheduled backups with a tested restore; a staging rehearsal for migrations; image scanning; alerts routed to a channel, exporters for nginx and Postgres.

02 A queue and a worker for the AI checks

So checks survive deploys and scale on their own; today they run in-process.

04 Frontend and edge hardening

httpOnly cookie sessions, CSP nonces, rate-limit windows shared across instances.

06 Data protection

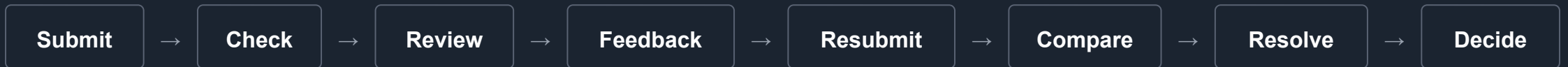
Retention and redaction for document text sent to the model; an in-region provider.

08 The admin epic

Oversight dashboard, AI health, audit feed, user management; role and routes reserved.



From submission to resolution: one traceable licensing workflow.



A working MVP built with production practices, in a three-day engineering constraint.

permitflow.space
github.com/SuhaasNv/permitflow